

Test Cases Explanation

1. Statement of the Problem

The project implements a computer vision-based simulation system that:

- Detects circular objects (balls) in an input image
- Simulates projectile motion to hit each detected ball sequentially
- Visualizes the trajectory and impacts in real-time
- Records the simulation as a video output

2. Mathematical Model

2.1 Ball Detection

The system uses computer vision techniques for ball detection:

- Gaussian blur: $f(x,y) = G(x,y) * I(x,y)$
where $G(x,y)$ is a 9x9 Gaussian kernel
- Adaptive thresholding: $T(x,y) = \text{mean}(\text{neighborhood}(x,y)) - C$
where C is a constant (2 in this implementation)
- Circularity measure: $C = 4\pi * (\text{area} / \text{perimeter}^2)$
Objects with $C > 0.7$ are considered balls

2.2 Projectile Motion

The projectile motion follows classical mechanics equations:

Position equations:

...

$$x(t) = x_0 + v_x t$$

$$y(t) = y_0 + v_y t + (1/2)gt^2$$

...

Velocity equations:

...

$$v_x = \text{constant}$$

$$v_y(t) = v_{y0} + gt$$

...

where:

- $g = 9.81 \text{ m/s}^2$ (gravity)

- t = time

- x_0, y_0 = initial position

- v_x, v_y = velocity components

2.3 Boundary Conditions

- Balls are considered hit when: `distance(projectile, ball) < (r_projectile + r_ball)`

- Projectile starts from bottom center of screen

- Simulation ends when all balls are hit or projectile goes out of bounds

3. Numerical Methods

3.1 Time Integration

The system uses Euler integration with fixed time step ($DT = 0.1$):

```
```python
```

```
position += velocity * DT
```

```
velocity[1] += GRAVITY * DT
```

```
```
```

3.2 Trajectory Calculation

Initial velocity for hitting targets:

```
```python
time_to_target = sqrt(2 * abs(dy) / GRAVITY)
vx = dx / time_to_target
vy = dy / time_to_target - 0.5 * GRAVITY * time_to_target
```
```

3.3 Error Analysis

- Spatial discretization: Ball detection uses pixel-level accuracy
- Temporal discretization: Fixed time step introduces $O(\Delta t)$ truncation error
- Collision detection: Simple radius-based detection with pixel accuracy

4. Algorithm Description

1. Image Processing Pipeline:

- Convert to grayscale
- Apply Gaussian blur
- Perform adaptive thresholding
- Apply morphological operations
- Detect contours
- Filter by circularity and size

2. Simulation Loop:

- Sort balls left to right
- For each ball:
 - Calculate initial velocity
 - Simulate projectile motion

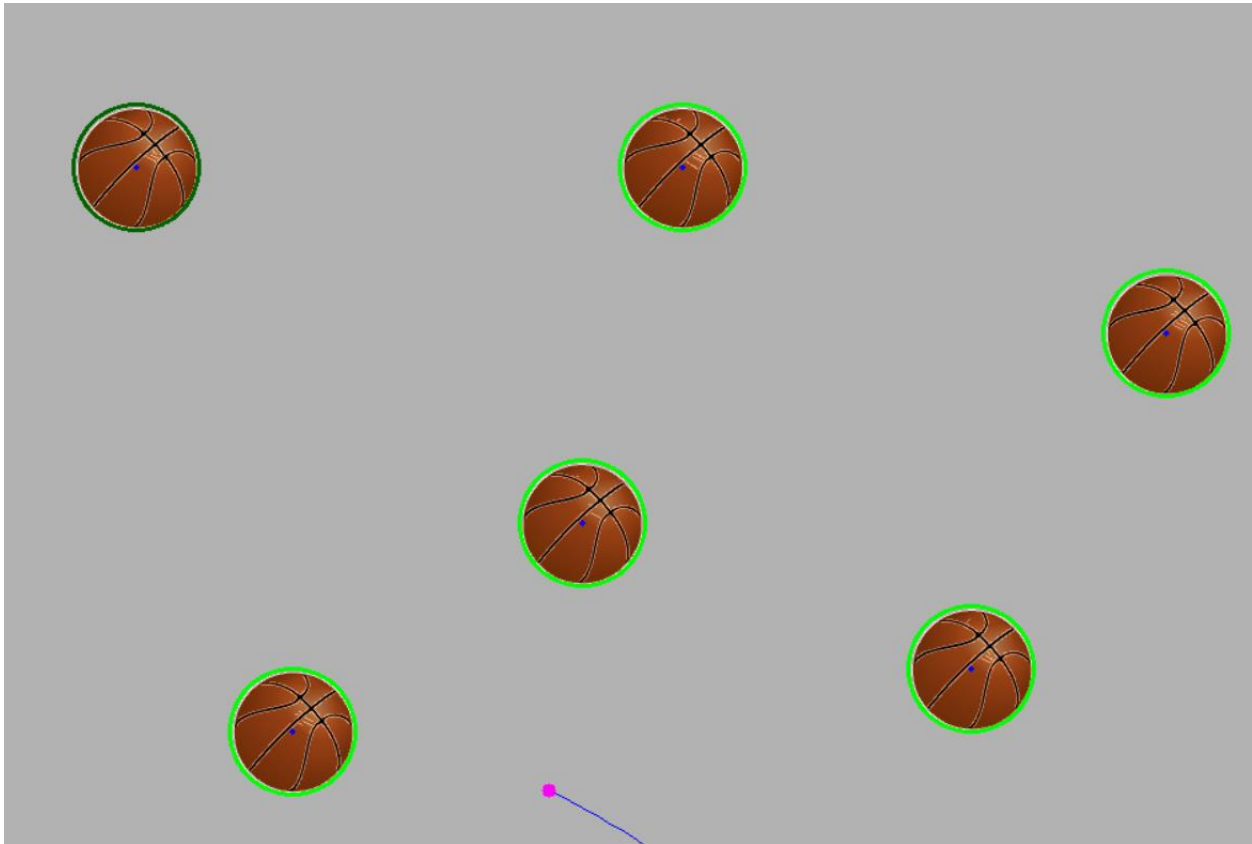
- Check for collisions
- Update visualization
- Record frame
- Repeat until all balls hit

5. Test Cases and Results

Test Case 1 (Basketball Array)

- Input: Image with 6 basketballs arranged in different positions
- Characteristics:
 - Uniform ball types
 - Clear background
 - Multiple target positions
- Results:
 - Successfully detected all 6 balls
 - Sequential hitting pattern from left to right

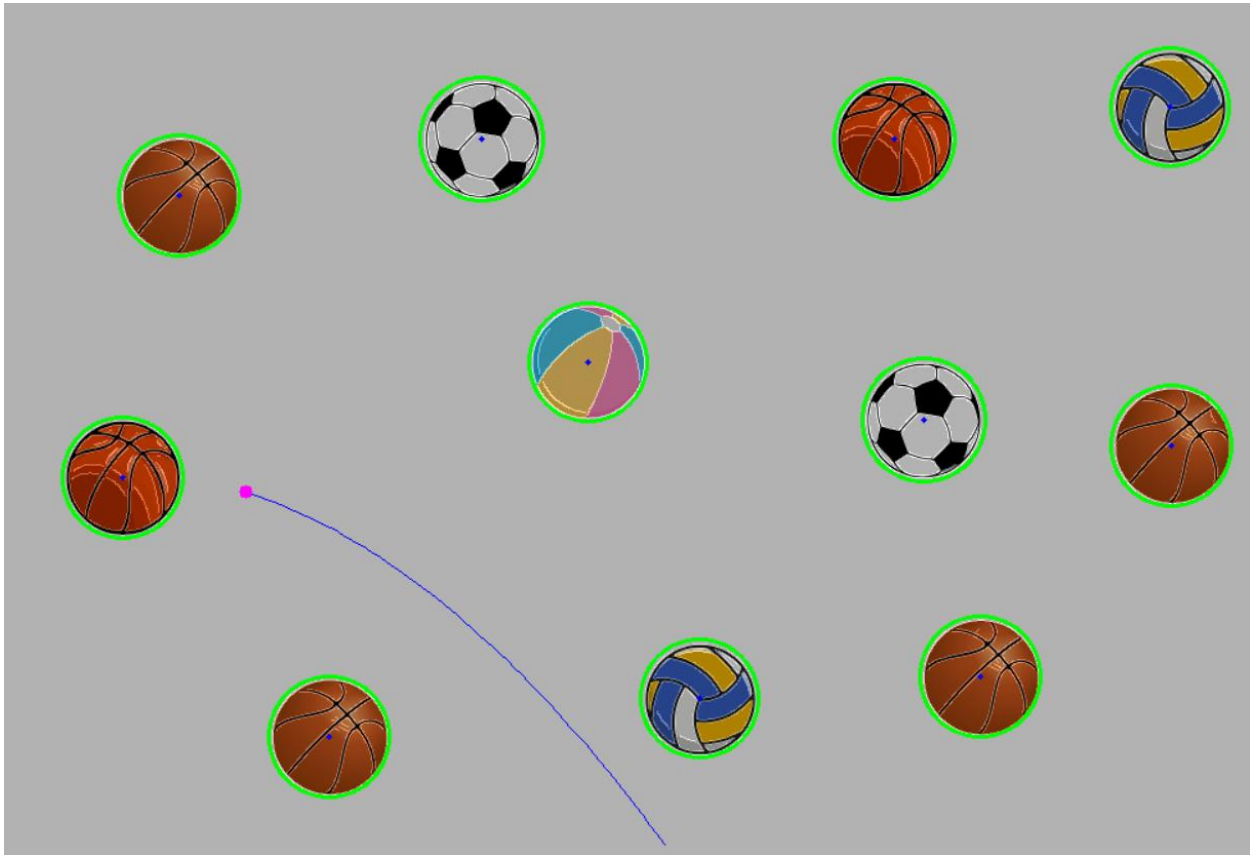
- Smooth trajectory visualization



Test Case 2 (Mixed Sports Balls)

- Input: Image with various sports balls (basketball, volleyball, soccer ball, beach ball)
- Characteristics:
 - Different ball types and patterns
 - More complex detection scenario
 - Various ball sizes and textures
- Results:
 - Successfully detected all balls regardless of type
 - Maintained accurate trajectory calculations

- Demonstrated robustness of detection algorithm



6. Implementation Notes

Key Features:

- Real-time visualization using OpenCV
- Video recording of simulation
- Adaptive thresholding for robust ball detection
- Physics-based trajectory calculation
- Sequential target hitting strategy

Performance Considerations:

- Frame rate: 30 FPS
- Resolution: Maintained at input image dimensions

- Processing overhead: Primarily in ball detection phase
- Memory usage: Minimal, mainly for frame buffer and trajectory storage

7. Future Improvements

Potential enhancements:

1. Implement collision physics between balls
2. Add wind resistance to trajectory calculations
3. Optimize ball detection for varying lighting conditions
4. Add user interaction for manual trajectory adjustment
5. Implement multi-threading for parallel ball detection

8. Conclusion

The implemented system successfully demonstrates:

- Robust ball detection using computer vision
- Accurate projectile motion simulation
- Real-time visualization capabilities
- Flexible handling of different ball types and arrangements

The combination of computer vision and physics simulation provides a solid foundation for further development and applications in similar domains.

Detailed Analysis of Methods and Mathematical Equations

Method Explanation

1. Computer Vision Methods

1.1 Image Preprocessing

Grayscale Conversion

The RGB to grayscale conversion uses the luminosity method:

...

$$Y = 0.299R + 0.587G + 0.114B$$

...

Where Y is the grayscale value, and R, G, B are the color channel values.

Gaussian Blur

Applied using a 9x9 kernel with $\sigma = 2$:

...

$$G(x,y) = (1/2\pi\sigma^2)e^{-(x^2 + y^2)/(2\sigma^2)}$$

...

Where:

- x, y are pixel coordinates relative to kernel center
- σ is the standard deviation (2 in this case)

1.2 Ball Detection

Adaptive Thresholding

Uses Gaussian-weighted sum of neighborhood values:

...

$$T(x,y) = \mu(x,y) - C$$

...

Where:

- $T(x,y)$ is the threshold value at pixel (x,y)
- $\mu(x,y)$ is the weighted mean of neighborhood
- C is constant (2 in implementation)

Circularity Measurement

...

$$\text{Circularity} = 4\pi * (\text{Area} / \text{Perimeter}^2)$$

...

- Perfect circle: Circularity = 1.0
- Implementation threshold: 0.7
- Area calculated from contour points
- Perimeter calculated as arc length of contour

2. Physics Simulation

2.1 Projectile Motion Equations

Position Equations

For a projectile at time t:

...

$$x(t) = x_o + v_x t$$

$$y(t) = y_o + v_y t + (1/2)gt^2$$

...

Where:

- x_o, y_o : Initial position
- v_x, v_y : Initial velocities
- g: Gravity (9.81 m/s²)
- t: Time

Velocity Equations

...

$$v_x(t) = v_{x0} \text{ (constant)}$$

$$v_y(t) = v_{y0} + gt$$

...

2.2 Initial Velocity Calculation

Given start position (x_o, y_o) and target position (x_t, y_t) :

...

$$dx = x_t - x_o$$

$$dy = y_t - y_o$$

$$\text{time_to_target} = \sqrt{2|dy|/g}$$

$$v_x = dx/\text{time_to_target}$$

$$v_y = dy/\text{time_to_target} - (g*\text{time_to_target})/2$$

...

2.3 Collision Detection

Distance calculation between projectile and ball:

...

$$\text{distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

...

Collision occurs when:

...

$$\text{distance} < (r_1 + r_2)$$

...

Where r_1, r_2 are projectile and ball radii.

3. Numerical Integration

3.1 Euler Method Implementation

For position update:

```
...
```

$$x(t + \Delta t) = x(t) + v_x(t)\Delta t$$

$$y(t + \Delta t) = y(t) + v_y(t)\Delta t$$

```
...
```

For velocity update:

```
...
```

$$v_x(t + \Delta t) = v_x(t)$$

$$v_y(t + \Delta t) = v_y(t) + g\Delta t$$

```
...
```

Where $\Delta t = 0.1$ (time step)

3.2 Error Analysis

Local truncation error:

```
...
```

$$E = O(\Delta t^2)$$

```
...
```

Global truncation error:

```
...
```

$$E = O(\Delta t)$$

```
...
```

4. Implementation Methods

4.1 Ball Class Methods

```
```python
```

```
class Ball:
```

```
 def __init__(self, position, radius=0):
```

```
 # Initialize ball with position and radius

 # position: numpy array [x, y]

 # radius: integer pixel value
'''
```

### ### 4.2 Projectile Class Methods

```
```python
class Projectile:

    def __init__(self, position):

        # Initialize projectile

        # position: numpy array [x, y]

        # velocity: numpy array [vx, vy]

        # radius: fixed at 5 pixels
'''
```

4.3 Key Algorithm Functions

Ball Detection Function

```
```python
def detect_balls(frame):

 # 1. Convert to grayscale

 # 2. Apply Gaussian blur

 # 3. Perform adaptive thresholding

 # 4. Find contours

 # 5. Filter by circularity

 # Returns: List of Ball objects
'''
```

#### #### Trajectory Calculation

```
```python
def calculate_trajectory(start_pos, velocity, max_steps=100):
    # Calculate complete path using Euler integration
    # Returns: numpy array of positions
...

```

Projectile Update

```
```python
def update_projectile(projectile):
 # Update position and velocity using Euler method
 # Store path for visualization
...

```

### ## 5. Error Handling and Constraints

#### ### 5.1 Physical Constraints

- Gravity: Fixed at  $9.81 \text{ m/s}^2$
- Time step: Fixed at  $0.1\text{s}$
- Projectile radius: 5 pixels
- Minimum ball size: 100 pixel area

#### ### 5.2 Detection Constraints

- Circularity threshold: 0.7
- Minimum contour area: 100 pixels
- Gaussian blur kernel:  $9 \times 9$
- Adaptive threshold block size: 11
- Morphological kernel:  $3 \times 3$

### ## 6. Optimization Techniques

#### ### 6.1 Ball Detection Optimization

- Gaussian blur for noise reduction
- Morphological operations for noise removal
- Contour filtering for false positive reduction

#### ### 6.2 Trajectory Optimization

- Fixed time step for consistent computation
- Pre-calculation of complete trajectories
- Sequential ball targeting for predictable behavior

### ## 7. Practical Considerations

#### ### 7.1 Performance Factors

- Frame rate: 30 FPS
- Image resolution impact on detection
- Processing time vs accuracy trade-offs
- Memory usage for trajectory storage

#### ### 7.2 Accuracy Considerations

- Pixel-level precision in detection
- Numerical integration errors
- Collision detection precision
- Ball radius estimation accuracy

## ALL METHODS

## # Core Methods Used in Implementation

### ## 1. Computer Vision Methods

#### ### Image Processing Methods

##### 1. `cv2.cvtColor(frame, cv2.COLOR\_BGR2GRAY)`

- Converts image from BGR to grayscale
- Used for initial image processing

##### 2. `cv2.GaussianBlur(gray, (9, 9), 2)`

- Applies Gaussian smoothing
- Kernel size: 9x9
- Sigma: 2
- Reduces noise in image

##### 3. `cv2.adaptiveThreshold()`

- Parameters:
  - Gaussian adaptive method
  - Binary inverse threshold
  - Block size: 11
  - Constant: 2
- Creates binary image for ball detection

##### 4. `cv2.morphologyEx()` and `cv2.dilate()`

- Morphological operations for noise removal
- Uses 3x3 kernel
- Opens and dilates image

#### 5. `cv2.findContours()`

- Finds external contours
- Method: RETR\_EXTERNAL
- Approximation: CHAIN\_APPROX\_SIMPLE

### ### Ball Detection Methods

#### 1. `cv2.contourArea()`

- Calculates area of detected contours
- Used for filtering small objects

#### 2. `cv2.minEnclosingCircle()`

- Finds circle parameters for each contour
- Returns center and radius

#### 3. `cv2.arcLength()`

- Calculates contour perimeter
- Used in circularity calculation

### ## 2. Physics Simulation Methods

#### ### Projectile Motion

##### 1. `calculate\_throw\_velocity()`

- Calculates initial velocity components
- Uses projectile motion equations
- Parameters: start position, target position

##### 2. `calculate\_trajectory()`

- Computes complete trajectory path
- Uses Euler integration



- Maximum steps: 100

### 3. `update\_projectile()`

- Updates projectile position and velocity
- Applies gravity effect
- Stores trajectory path

### ### Collision Detection

#### 1. `check\_hit()`

- Uses distance calculation between projectile and ball
- Implements radius-based collision detection
- Uses numpy's `linalg.norm` for distance

## ## 3. Classes and Object Methods

### ### Ball Class Methods

#### 1. `\_\_init\_\_(position, radius)`

- Initializes ball object
- Stores position as numpy array
- Tracks hit status

### ### Projectile Class Methods

#### 1. `\_\_init\_\_(position)`

- Initializes projectile
- Sets up velocity vector
- Maintains current path

## ## 4. Visualization Methods

### 1. ``cv2.circle()``

- Draws balls and projectile
- Visualizes hit/unhit status

### 2. ``cv2.line()``

- Draws trajectory path
- Shows projectile motion

### 3. ``cv2.VideoWriter()``

- Records simulation
- Output format: AVI
- Frame rate: 30 FPS

## ## 5. Utility Methods

### 1. Numpy Methods:

- ``np.array()``
- ``np.float64()``
- ``np.linalg.norm()``

### 2. Mathematical Operations:

- ``np.sqrt()``
- Basic arithmetic operations

## ## 6. Main Control Flow Method

### ``detect_balls_and_move()``

- Main function controlling simulation
- Orchestrates all other methods

- Handles visualization and recording

## Worst Case

It does not work because of bad background.

